The Queue ADT

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Definition
2. Operations
3. Applications
4. Array Implementation
5. Linked-List Implementation
6. Priority Queue
7. Double-Ended Queue (Deque)

Definition and Properties

- A **queue** is a linear ADT supporting access at two ends: **front** (dequeue) and **rear** (enqueue).
- **FIFO** discipline: First-In, First-Out.
- **State:** an ordered sequence $\langle a_1, \ldots, a_k \rangle$ where a_1 is at the front and a_k at the rear.
- **Safety:** operations defined only when preconditions hold (e.g., non-empty for `front`/`dequeue`).
- **Capacity:** bounded (array-based) or unbounded (linked list); arrays typically use a **circular buffer**.

Basic Operations

- `enqueue()`: insert item at the rear.
- `dequeue()`: remove and return the front element.
- `front()`: return (peek) front element without removing.
- `isEmpty()`: true if queue has no elements.
- `size()`: number of elements.

Exceptions

- **Underflow:** calling `dequeue`/`front` on empty queue.
- **Overflow:** enqueuing into a full bounded queue (array capacity reached).

Applications of Queues

- **Scheduling & buffering:** CPU scheduling, I/O buffers, print spooling, network packets.
- **Level-order processing: BFS** on graphs/trees (shortest paths in unweighted graphs).
- **Producer–Consumer** pipelines: decouple production and consumption rates.
- **Simulation:** bank/checkout lines, call centers, discrete-event simulation.
- **Rate limiting:** token buckets, sliding windows.

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`	void	$Q = \langle 3, 9, 4 \rangle$
7	`size()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`	void	$Q = \langle 3, 9, 4 \rangle$
7	`size()`	3	$Q = \langle 3, 9, 4 \rangle$
8	`dequeue()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`	void	$Q = \langle 3, 9, 4 \rangle$
7	`size()`	3	$Q = \langle 3, 9, 4 \rangle$
8	`dequeue()`	3	$Q = \langle 9, 4 \rangle$
9	`dequeue()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`	void	$Q = \langle 3, 9, 4 \rangle$
7	`size()`	3	$Q = \langle 3, 9, 4 \rangle$
8	`dequeue()`	3	$Q = \langle 9, 4 \rangle$
9	`dequeue()`	9	$Q = \langle 4 \rangle$
10	`isEmpty()`		

Example: Simple Queue Operations (Trace)

Initial state: $Q = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`enqueue(7)`	void	$Q = \langle 7 \rangle$
2	`enqueue(3)`	void	$Q = \langle 7, 3 \rangle$
3	`front()`	7	$Q = \langle 7, 3 \rangle$
4	`enqueue(9)`	void	$Q = \langle 7, 3, 9 \rangle$
5	`dequeue()`	7	$Q = \langle 3, 9 \rangle$
6	`enqueue(4)`	void	$Q = \langle 3, 9, 4 \rangle$
7	`size()`	3	$Q = \langle 3, 9, 4 \rangle$
8	`dequeue()`	3	$Q = \langle 9, 4 \rangle$
9	`dequeue()`	9	$Q = \langle 4 \rangle$
10	`isEmpty()`	`false`	$Q = \langle 4 \rangle$

Queue via Array: Circular Buffer Idea

- Use a fixed-size array of capacity C and two indices: `front` (points to current front) and `rear` (next insertion position).
- Indices **wrap around** with modulo C: `(index+1) % C`.
- **Empty** if `size` $= 0$. **Full** if `size` $= C$.
- Avoid ambiguity by tracking `size` explicitly (or leaving one slot empty).

Circular Queue: Operations

- **enqueue():** if not full, write `A[rear]=x`; update `rear=(rear+1)%C`; `size=(size+1)`.
- **dequeue():** if not empty, read `A[front]`; update `front=(front+1)%C`; `size=(size-1)`.
- **front():** return `A[front]` if not empty.
- **isEmpty():** `size==0`, **isFull():** `size==C`.

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`	void	1	0	[_, 2, 3, 4, 5]
7	`enqueue(6)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`	void	1	0	[_, 2, 3, 4, 5]
7	`enqueue(6)`	void	1	1	[6, 2, 3, 4, 5]
8	`dequeue()`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`	void	1	0	[_, 2, 3, 4, 5]
7	`enqueue(6)`	void	1	1	[6, 2, 3, 4, 5]
8	`dequeue()`	2	2	1	[6, _, 3, 4, 5]
9	`enqueue(6)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`	void	1	0	[_, 2, 3, 4, 5]
7	`enqueue(6)`	void	1	1	[6, 2, 3, 4, 5]
8	`dequeue()`	2	2	1	[6, _, 3, 4, 5]
9	`enqueue(6)`	void	2	2	[6, 6, 3, 4, 5]
10	`enqueue(5)`				

Example 1: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(1)`	void	0	1	[1, _, _, _, _]
2	`enqueue(2)`	void	0	2	[1, 2, _, _, _]
3	`enqueue(3)`	void	0	3	[1, 2, 3, _, _]
4	`dequeue()`	1	1	3	[_, 2, 3, _, _]
5	`enqueue(4)`	void	1	4	[_, 2, 3, 4, _]
6	`enqueue(5)`	void	1	0	[_, 2, 3, 4, 5]
7	`enqueue(6)`	void	1	1	[6, 2, 3, 4, 5]
8	`dequeue()`	2	2	1	[6, _, 3, 4, 5]
9	`enqueue(6)`	void	2	2	[6, 6, 3, 4, 5]
10	`enqueue(5)`	**Overflow**	2	2	[6, 6, 3, 4, 5] (unchanged)

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`	20	2	3	[_, _, 30, _, _]
6	`enqueue(40)`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`	20	2	3	[_, _, 30, _, _]
6	`enqueue(40)`	void	2	4	[_, _, 30, 40, _]
7	`dequeue()`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`	20	2	3	[_, _, 30, _, _]
6	`enqueue(40)`	void	2	4	[_, _, 30, 40, _]
7	`dequeue()`	30	3	4	[_, _, _, 40, _]
8	`dequeue()`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`	20	2	3	[_, _, 30, _, _]
6	`enqueue(40)`	void	2	4	[_, _, 30, 40, _]
7	`dequeue()`	30	3	4	[_, _, _, 40, _]
8	`dequeue()`	40	4	4	[_, _, _, _, _]
9	`dequeue()`				

Example 2: Circular Queue (Trace, $C = 5$)

Start: `front` $= 0$, `rear` $= 0$, `size` $= 0$

Step	Operation	Return	front	rear	Queue Snapshot
1	`enqueue(10)`	void	0	1	[10, _, _, _, _]
2	`enqueue(20)`	void	0	2	[10, 20, _, _, _]
3	`enqueue(30)`	void	0	3	[10, 20, 30, _, _]
4	`dequeue()`	10	1	3	[_, 20, 30, _, _]
5	`dequeue()`	20	2	3	[_, _, 30, _, _]
6	`enqueue(40)`	void	2	4	[_, _, 30, 40, _]
7	`dequeue()`	30	3	4	[_, _, _, 40, _]
8	`dequeue()`	40	4	4	[_, _, _, _, _]
9	`dequeue()`	**Underflow**	4	4	[_, _, _, _, _] (unchanged)

Array Queue: Complexity & Pros/Cons

- **Time Complexity:** `enqueue`, `dequeue`, `front`, `isEmpty`, `size` are all $\Theta(1)$.
- **Space:** $\Theta(C)$ contiguous memory.

Pros

- Simple, constant-time operations; excellent cache locality.

Cons

- Fixed capacity unless resized; resizing is $O(n)$.
- Need wrap-around logic (off-by-one pitfalls).

Queue via Linked List: Idea

- Maintain pointers to both `head` (front) and `tail` (rear).
- Each node stores a value and pointer to next node.
- **Empty** when `head = nullptr` (`tail` also `nullptr`).
- **enqueue:** insert at tail in $\Theta(1)$. **dequeue:** remove from head in $\Theta(1)$.

Linked-List Queue: Complexity & Pros/Cons

- **Time Complexity:** $\Theta(1)$ for `enqueue`, `dequeue`, `front`, `isEmpty`, `size`.
- **Space:** $\Theta(n)$ nodes, extra pointer per node.

Pros

- Unbounded growth; no fixed capacity constraints.
- No wrap-around logic; straightforward ends.

Cons

- Extra allocation/deallocation overhead; poorer cache locality.
- Pointer memory overhead.

Priority Queue: Concept

- A **Priority Queue (PQ)** stores elements each with a **priority** (key); removal returns the element with **highest** (max-PQ) or **lowest** (min-PQ) priority.
- Abstract operations:
  - `insert(x, key)` add an item with priority `key`.
  - `top()` (a.k.a. `findMin`/`findMax`) peek best-priority item.
  - `extract()` remove & return best-priority item.
  - Optional: `decreaseKey`/`increaseKey`, `merge`.
- **Ordering by priority**, not arrival time (not FIFO).
- Common Implementations: **Binary heap**

Deque: Concept

- A **Double-Ended Queue (Deque)** supports insertion and deletion at **both ends**.
- Abstract operations:
  - `pushFront(x)`
  - `pushBack(x)`
  - `popFront()`
  - `popBack()`
  - `front()`, `back()`
  - `isEmpty()`, `size()`
- Implementations: circular array (block-chunked for growth), doubly linked list.
- Typical costs: all above ops in $\Theta(1)$ amortized (array) or worst-case (list).